Book Store QA

Testing manual — unit, API and UI

Generated 27 August 2026, 13:49 · commit 8857af6
Vitest · Karate · Playwright · GitHub Actions

1. Testing architecture

Three layers, run in a strict sequence. Each is cheaper and more certain than the one after it, so the pipeline learns the cheapest thing first and stops as soon as it knows enough.

Layer	Tool	Scope	Needs a target URL?	Fails the stage below it
Unit	Vitest	Pure logic in `playwright/src` — no browser, no network	No	Skips API **and** UI
API	Karate (Java 17 + Maven)	Book Store REST contract, per scenario	Yes	Skips UI
UI	Playwright	Browser journeys, one leg per browser	Yes	—

```
0. unit (Vitest)  →  api (Karate)  →  ui (Playwright, one leg per browser)
```

Why unit tests run once, not per environment. Unit tests assert pure logic — they never open a browser, never make a request, never touch demoqa.com. Their answer therefore *cannot* differ between dev, release and production. Running them inside each stage bought three identical answers to the same question, and made the API suite wait behind a redundant `npm ci` every time. They now run once, straight after the run configuration is resolved, as job `0. Unit tests (Vitest)`.

Because they run once, they have to gate *every* way into the chain. `dev` inherits that through ordinary job dependencies, but `release` and `production` are entry points in their own right and their conditions use a status function, which opts out of the implicit "all dependencies succeeded" rule. Both therefore name the result explicitly — without that, starting a run from `release` would walk straight past the gate.

2. Local execution

Three parameters run through everything: **environment** (which host), **suite** (smoke or regression) and **browser**.

> **Windows PowerShell.** Run the lines in each block one at a time. PowerShell 5.1 has no `&&`, so joining them fails with "*'&&' is not a valid statement separator*". Use `;` to chain, or `; if ($?) { … }` to stop on the first failure. Git Bash and PowerShell 7+ accept `&&` as written. PowerShell also has no inline `VAR=value command` prefix — use `$env:VAR='…'` on its own statement.

Unit tests and coverage (Vitest)

```
cd playwright
npm ci
npm run test:unit                 # tests only
npm run test:unit -- --coverage   # tests + coverage gate
npm run test:unit:watch           # re-run on save while developing
```

Coverage is scoped to the code unit tests are responsible for. `src/pages/**` and `src/utils/api-client.ts` are excluded by rule, because the Playwright suite and the live API are what exercise them — counting them here would measure the absence of Playwright rather than the quality of these tests.

Metric	Covered	Total	%	Floor
Statements	2	2	100%	80%
Branches	1	1	100%	80%
Functions	1	1	100%	80%
Lines	2	2	100%	80%

Vitest fails the run itself when any metric drops below the floor, so the gate is the runner's and not a later script's reading of a report. Read the figure as "*the pure logic is covered*", not "*the project is covered*" — the covered surface is deliberately small.

API tests (Karate)

Requires **Java 17** and Maven 3.8+. Karate's GraalJS engine does not support JDK 18+ — on a newer JDK the suite hangs silently instead of failing, so check `java -version` first.

```
cd karate
```

Quote any `-D` containing a dot on PowerShell: it ends a parameter name at the first `.`, so `-Dkarate.env=dev` arrives at Maven split in two and fails with "*Unknown lifecycle phase '.env=dev'*".

Dev — smoke:

```
mvn test -Dtest=SmokeTest -Dkarate.env=dev
```

```
mvn test '-Dtest=SmokeTest' '-Dkarate.env=dev'
```

Release — full regression:

```
mvn test -Dtest=BookStoreApiTest -Dkarate.env=release
```

```
mvn test '-Dtest=BookStoreApiTest' '-Dkarate.env=release'
```

SCREENSHOT PLACEHOLDER

Karate regression against release

Capture with:

mvn test -Dtest=BookStoreApiTest -Dkarate.env=release

docs/images/local-karate-release.png

Production — smoke, with an explicit host:

```
mvn test -Dtest=SmokeTest -Dkarate.env=production -DbaseUrl=https://demoqa.com
```

```
mvn test '-Dtest=SmokeTest' '-Dkarate.env=production' '-DbaseUrl=https://demoqa.com'
```

SCREENSHOT PLACEHOLDER

Karate smoke against production

Capture with:

mvn test -Dtest=SmokeTest -Dkarate.env=production

docs/images/local-karate-production.png

`-Dtest` picks the runner class and therefore the tag set: `SmokeTest` runs `@smoke`, `BookStoreApiTest` runs everything. Reports land in `karate/target/karate-reports/karate-summary.html`.

UI tests (Playwright)

```
cd playwright
npx playwright install chromium              # or: firefox webkit msedge
```

Dev — smoke on chromium:

```
BASE_URL=https://demoqa.com npx playwright test --project=chromium --grep @smoke
```

```
$env:BASE_URL='https://demoqa.com'; npx playwright test --project=chromium --grep @smoke
```

Release — regression across every browser:

```
BASE_URL=https://demoqa.com npx playwright test --project=chromium --project=firefox --
project=webkit --project=msedge
```

```
$env:BASE_URL='https://demoqa.com'; npx playwright test --project=chromium --project=firefox --
project=webkit --project=msedge
```

SCREENSHOT PLACEHOLDER

Playwright regression across browsers

Capture with:

npx playwright test --project=chromium --project=firefox --project=webkit --project=msedge

docs/images/local-playwright-release.png

Production — smoke on Edge:

```
BASE_URL=https://demoqa.com npx playwright test --project=msedge --grep @smoke
```

```
$env:BASE_URL='https://demoqa.com'; npx playwright test --project=msedge --grep @smoke
```

SCREENSHOT PLACEHOLDER

Playwright smoke against production

Capture with:

npx playwright test --project=msedge --grep @smoke

docs/images/local-playwright-production.png

UI mode — pick and re-run tests interactively:

```
npx playwright test --ui
npm run test:headed     # or just watch chromium drive
npm run report          # open the last HTML report
```

Edge is a branded channel rather than a bundled engine, so `npx playwright install` alone does not fetch it — it has to be named. `playwright test` has no `--channel` flag either, so the config models Edge as a project pinning `channel: 'msedge'`; one `--project` argument then selects any of the four.

The whole sequence in one line

Unit → API → UI, stopping at the first failure:

```
cd playwright && npm run test:unit -- --coverage && cd ../karate && mvn test -Dtest=SmokeTest -Dkarate.env=dev && cd ../playwright && BASE_URL=https://demoqa.com npx playwright test --project=chromium --grep @smoke
```

```
cd playwright; if ($?) { npm run test:unit -- --coverage }; if ($?) { cd ../karate }; if ($?) { mvn test '-Dtest=SmokeTest' '-Dkarate.env=dev' }; if ($?) { cd ../playwright }; if ($?) { $env:BASE_URL='https://demoqa.com'; npx playwright test --project=chromium --grep @smoke }
```

3. GitHub Actions pipeline

Run it from **Actions** → **QA Automation** → **Run workflow**. Four things decide what happens, and the first of them is the branch:

Input	Options	Effect
Use workflow from	`eyter_dev`, `release`, `main`	Chooses the entry stage — this is why there is no environment dropdown
Test suite	`smoke`, `regression`	`-Dtest=SmokeTest` / `BookStoreApiTest`, and `--grep @smoke`
Playwright browser	`chromium`, `firefox`, `webkit`, `msedge`, `all browsers`	One matrix leg each; `all browsers` runs four in parallel
Promote	checkbox, off by default	Whether the run continues past its entry stage

Scenario A — running from `eyter_dev`

The full chain. Unit tests gate everything; the dev stage runs against the dev host; if it is green and *Promote* is ticked, the tested commit is fast-forwarded onto `release`, the release stage runs, and the same happens onto `main` before production is deployed and verified.

```
0. unit → 1. dev → promote → 2. release → promote → 3. deploy → verify
```

With *Promote* unticked the run stops after the dev stage: nothing is promoted, no branch moves, nothing is deployed.

Scenario B — running from `release` or `main`

The branch is the entry point, so earlier stages are skipped and the run starts where you pointed it. **Promotion is bound to `eyter_dev`**: ticking *Promote* on a run started from `release`, `main` or a feature branch fails the run in seconds, before any suite starts, with

> Promotion is only allowed when triggered from the eyter_dev branch.

That guard matters most in the case that looks harmless — a feature branch enters at the dev stage exactly like `eyter_dev` does, so without it a green feature-branch run would push its own commit straight at `release`.

Two things to know before running from `main`: entering at stage 3 **deploys**, promote or not, because `main` has nowhere further to promote to; and a `Pipeline result` job fails the run if production was deployed but its verification did not pass, so a deploy can never report green unverified.

SCREENSHOT PLACEHOLDER

Scenario B - single stage from release or main

Capture with:

Actions > QA Automation > Run workflow, from release or main

docs/images/ci-scenario-b-release-main.png

What the run summary shows

Every job writes its results to the run summary page, so a failure is readable without downloading an artifact: the Vitest coverage table with a tick or cross per metric, then a Karate table and a Playwright table per stage — totals, a row per suite with pass/fail/skip counts and durations, and, when something fails, a second table naming each failing test with the first line of its error.

SCREENSHOT PLACEHOLDER

The GitHub job summary

Capture with:

The Summary tab of any completed run

docs/images/ci-job-summary.png

Live results

Captured by running each suite on the machine that built this report, so the commands above are shown working rather than asserted to work.

Suite	Command	Result	Duration
Unit (Vitest)	`npm run test:unit -- --coverage`	passed	2.6s
API (Karate)	`mvn -B test -Dtest=SmokeTest -Dkarate.env=dev`	passed	21.5s
UI (Playwright)	`npx playwright test --project=chromium --grep @smoke`	passed	17.6s

Measured coverage

Metric	Covered	Total	%
Statements	2	2	100.0%
Branches	1	1	100.0%
Functions	1	1	100.0%
Lines	2	2	100.0%

Unit (Vitest) — exit 0 · 2.6s

```
> bookstore-web-automation@1.0.0 test:unit
> vitest run --coverage
 RUN  v4.1.11 C:/Users/g6k78/qa-assessment/playwright
      Coverage enabled with v8
 ✓ src/utils/test-data.test.ts (7 tests) 13ms
 Test Files  1 passed (1)
      Tests  7 passed (7)
   Start at  13:49:17
   Duration  444ms (transform 52ms, setup 0ms, import 81ms, tests 13ms, environment 0ms)
JUNIT report written to C:/Users/g6k78/qa-assessment/playwright/test-results/unit-junit.xml
 % Coverage report from v8
------------------------------ Coverage summary ------------------------------
Statements   : 100% ( 2/2 )
Branches     : 100% ( 1/1 )
Functions    : 100% ( 1/1 )
Lines        : 100% ( 2/2 )
------------------------------------------------------------------------------
```

API (Karate) exit 0 · 21.5s

```
... 157 earlier lines omitted ...
-------------------------------------------------------
13:49:39.082 [pool-3-thread-1] INFO  c.i.k.Suite - <<pass>> feature 1 of 5 (4 remaining)
classpath:bookstore/account/account.feature
-------------------------------------------------------
feature: classpath:bookstore/books/books.feature
scenarios:  1 | passed:  1 | failed:  0 | time: 5,5334
-------------------------------------------------------
13:49:39.599 [pool-3-thread-1] INFO  c.i.k.Suite - <<pass>> feature 2 of 5 (3 remaining)
classpath:bookstore/books/books.feature
Karate version: 1.4.1 | env: dev
=======================================================
elapsed:  11,16 | threads:    3 | thread time: 10,58
features:     2 | skipped:    3 | efficiency: 0,32
scenarios:    2 | passed:     2 | failed: 0
=======================================================
HTML report: (paste into browser to view) | Karate version: 1.4.1 | env: dev
file:///C:/Users/g6k78/qa-assessment/karate/target/karate-reports/karate-summary.html
=======================================================================
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 12.02 s -- in
bookstore.SmokeTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  17.587 s
[INFO] Finished at: 2026-08-27T13:49:40+03:00
[INFO] ------------------------------------------------------------------------
```

UI (Playwright) exit 0 · 17.6s

```
Running 5 tests using 4 workers
  ok 1 [chromium] › tests\login.spec.ts:27:7 › Login @smoke › protects the profile page from anonymous access (8.8s)
  ok 3 [chromium] › tests\registration.spec.ts:11:7 › Registration @smoke › the registration form is reachable from login and is captcha-protected (8.9s)
  ok 5 [chromium] › tests\registration.spec.ts:32:7 › Registration @smoke › the API rejects a password that violates the documented policy (316ms)
  ok 4 [chromium] › tests\login.spec.ts:4:7 › Login @smoke › rejects unknown credentials with a visible error and no session (10.2s)
  ok 2 [chromium] › tests\login.spec.ts:19:7 › Login @smoke › rejects a valid username with the wrong password (12.6s)
  5 passed (14.6s)
```